

Arys Garage Pvt Ltd

Real-Time Vehicle Telemetry & Data Logging System

Embedded Firmware Assignment Report

Submitted By

M Mohammed Riyaz

Role Applied

Embedded Integration Engineer

Assignment

Arys Garage – Assignment Based Evaluation – MAY 2026

Hardware Platform

STM32F401RE (NUCLEO-F401RE)

Software Stack:

Embedded C, STM32CubeIDE, STM32CubeMX, FreeRTOS

Submission Date

24 May 2026

1. Project Overview

1.1 Objective

The objective of this project is to design and implement a Real-Time Vehicle Telemetry & Data Logging System using the STM32F401RE microcontroller and FreeRTOS. The system is intended to collect, process, log, and stream telemetry information from multiple vehicle subsystems in real time.

The telemetry system performs:

- Sensor data acquisition
- Telemetry processing
- CSV logging
- Wireless telemetry streaming
- Fault monitoring and recovery
- Real-time task scheduling using FreeRTOS

The project architecture is designed similar to motorsport telemetry and automotive embedded systems.

2. System Requirements

The telemetry system was designed to:

- Acquire IMU, GPS, wheel speed, RPM, and CAN data
- Log telemetry data with timestamps
- Stream live telemetry to a monitoring interface
- Detect and recover from failures
- Run independent concurrent tasks using FreeRTOS
- Maintain deterministic timing behavior

3. Hardware Components

Component	Purpose
STM32F401RE (NUCLEO-F401RE)	Main microcontroller
MPU6050 IMU	Acceleration & Gyroscope sensing
Neo-6M GPS	Vehicle position and speed
MCP2515 CAN Module	CAN bus communication
ESP32 Module	WiFi/BLE telemetry streaming
SD Card Module	Telemetry data storage
UART Debug Interface	Serial telemetry monitoring

4. Embedded Firmware Source Code

4.1 Firmware Architecture

The firmware is structured using modular FreeRTOS tasks. Each subsystem operates independently and communicates through queues and shared telemetry structures.

Main firmware modules:

- Sensor acquisition drivers
- FreeRTOS scheduler
- Telemetry processing engine
- CSV logger
- Fault monitoring module
- CAN communication layer
- Wireless telemetry streaming layer

4.2 Firmware Features

Sensor Acquisition

- IMU data acquisition through I2C
- GPS data parsing through UART
- CAN frame reception through SPI MCP2515
- RPM and wheel speed monitoring

RTOS-Based Scheduling

- Multi-tasking using FreeRTOS
- Queue-based communication
- Priority scheduling
- Independent timing control

Telemetry Processing

- G-force computation
- Distance estimation
- Vehicle speed calculation
- Heading estimation
- Fault detection

Logging System

- Timestamped CSV logging
- Fault log recording
- SD card write monitoring
- Recovery and retry mechanisms

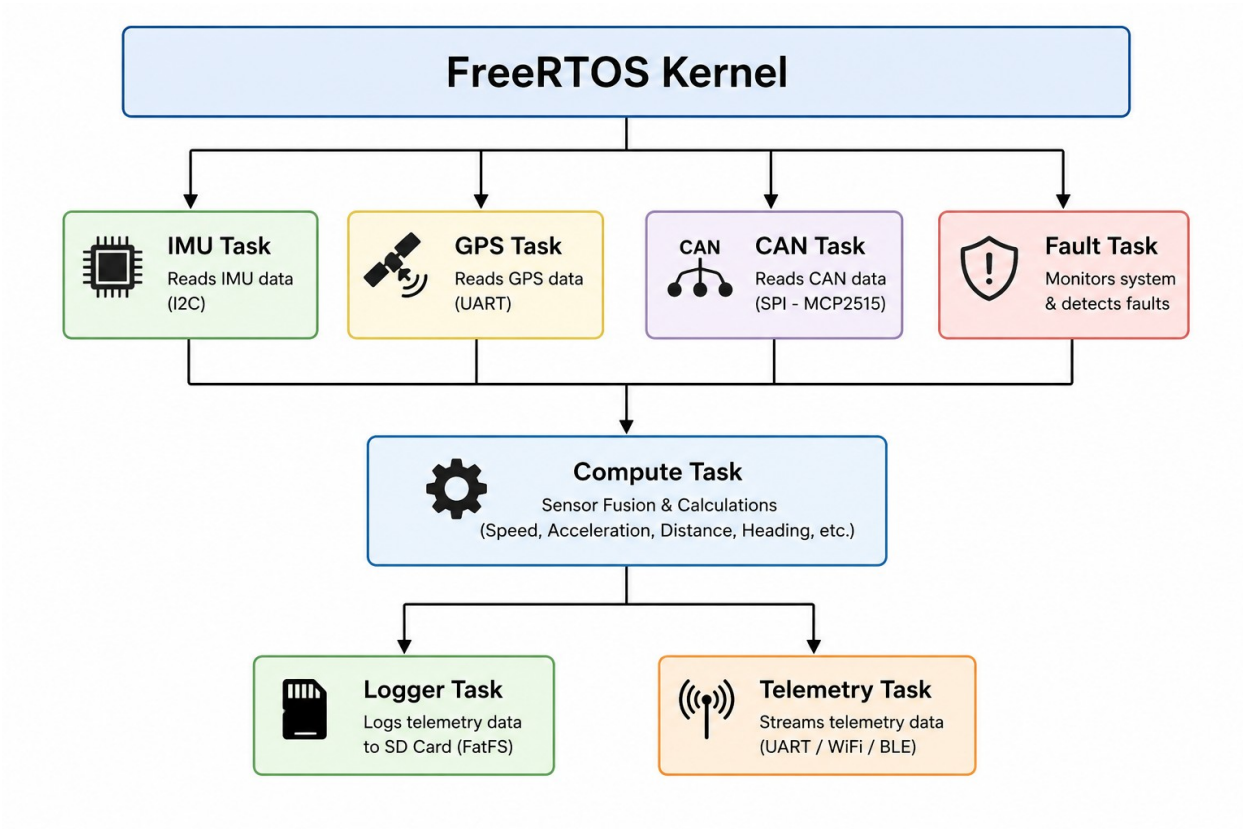
Telemetry Streaming

- UART streaming
- ESP32 wireless telemetry support
- Real-time monitoring output

5. RTOS Task Architecture Diagram

5.1 RTOS Task Structure

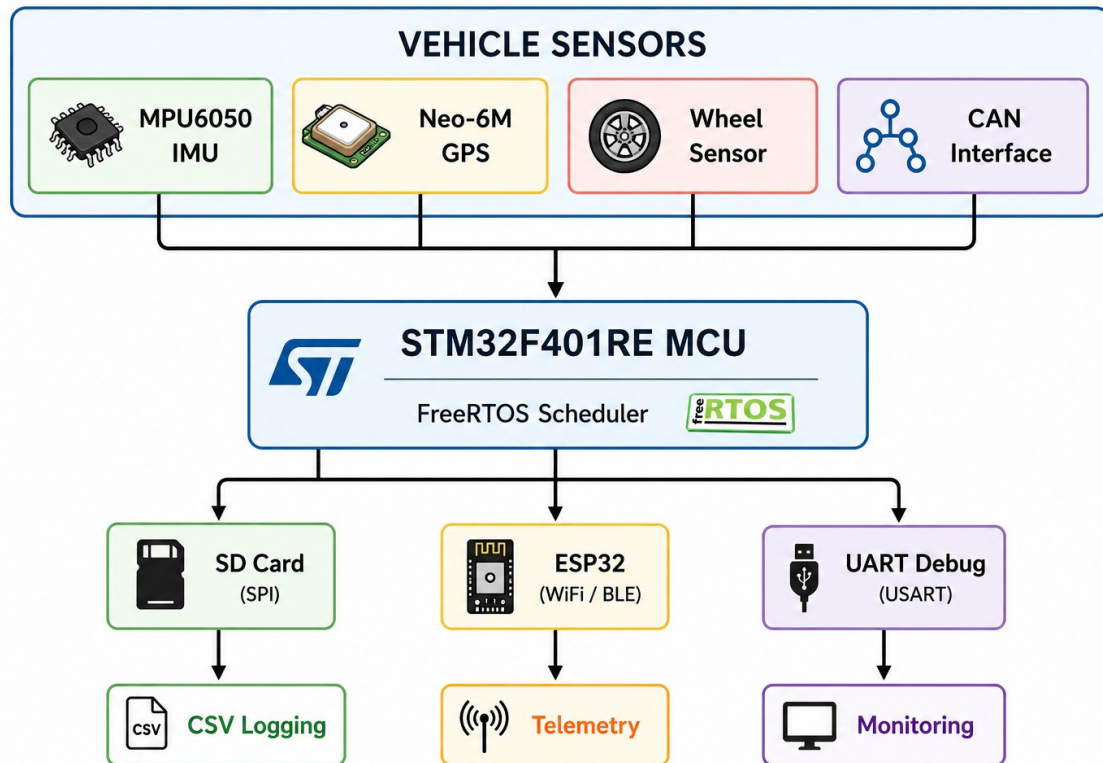
The telemetry system uses multiple concurrent FreeRTOS tasks.



5.2 Task Timing

Task	Frequency
IMU Task	50 Hz
GPS Task	1 Hz
CAN Task	5 Hz
Compute Task	10 Hz
Logger Task	10 Hz
Fault Monitor Task	1 Hz

6. System Block Diagram



7. Sensor Interface Schematics

7.1 MPU6050 IMU Interface

MPU6050 Pin	STM32F401RE Pin
VCC	3.3V
GND	GND
SDA	PB9 (I2C1 SDA)
SCL	PB8 (I2C1 SCL)

7.2 Neo-6M GPS Interface

GPS Pin	STM32F401RE Pin
TX	PA10 (USART1 RX)
RX	PA9 (USART1 TX)
VCC	3.3V
GND	GND

7.3 MCP2515 CAN Interface

MCP2515 Pin	STM32F401RE Pin
MOSI	PA7
MISO	PA6
SCK	PA5
CS	PB6
INT	PB5

7.4 ESP32 Telemetry Interface

ESP32 Pin	STM32F401RE Pin
RX	PA2 (USART2 TX)
TX	PA3 (USART2 RX)
GND	GND

8. Pseudocode

8.1 Main Firmware Flow

```
Initialize System Clock
Initialize GPIO
Initialize UART
Initialize SPI
Initialize I2C
Initialize SD Card
Initialize CAN Module
Initialize Sensors
Initialize FreeRTOS Queues
Create RTOS Tasks
Start Scheduler
```

8.2 IMU Task Pseudocode

```
while(1)
{
    Read MPU6050 Data
    Store Accelerometer Values
    Store Gyroscope Values
    Send Data To Queue
    Delay 20 ms
}
```

8.3 GPS Task Pseudocode

```
while(1)
{
    Read NMEA String
    Parse Latitude
    Parse Longitude
    Parse Speed
    Update GPS Structure
    Delay 1000 ms
}
```

8.4 CAN Task Pseudocode

```
while(1)
{
    Read CAN Frame
    Extract RPM
    Extract Wheel Speed
}
```

```
    Update CAN Data Structure  
    Delay 200 ms  
}
```

8.5 Logger Task Pseudocode

```
while(1)  
{  
    Collect Latest Telemetry  
    Convert To CSV Format  
    Write To SD Card  
    Stream UART Output  
    Delay 100 ms  
}
```

8.6 Fault Monitor Task Pseudocode

```
while(1)  
{  
    Check GPS Timeout  
    Check CAN Timeout  
    Check IMU Status  
    Check SD Write Status  
  
    If Fault Detected:  
        Raise Alarm  
        Attempt Recovery  
  
    Delay 1000 ms  
}
```

9. Logged Telemetry Data

The telemetry system continuously logs runtime data into CSV format.

9.1 Logged Parameters

- Timestamp
- Engine RPM
- Vehicle Speed
- Lateral G-force
- Vertical G-force
- GPS Latitude

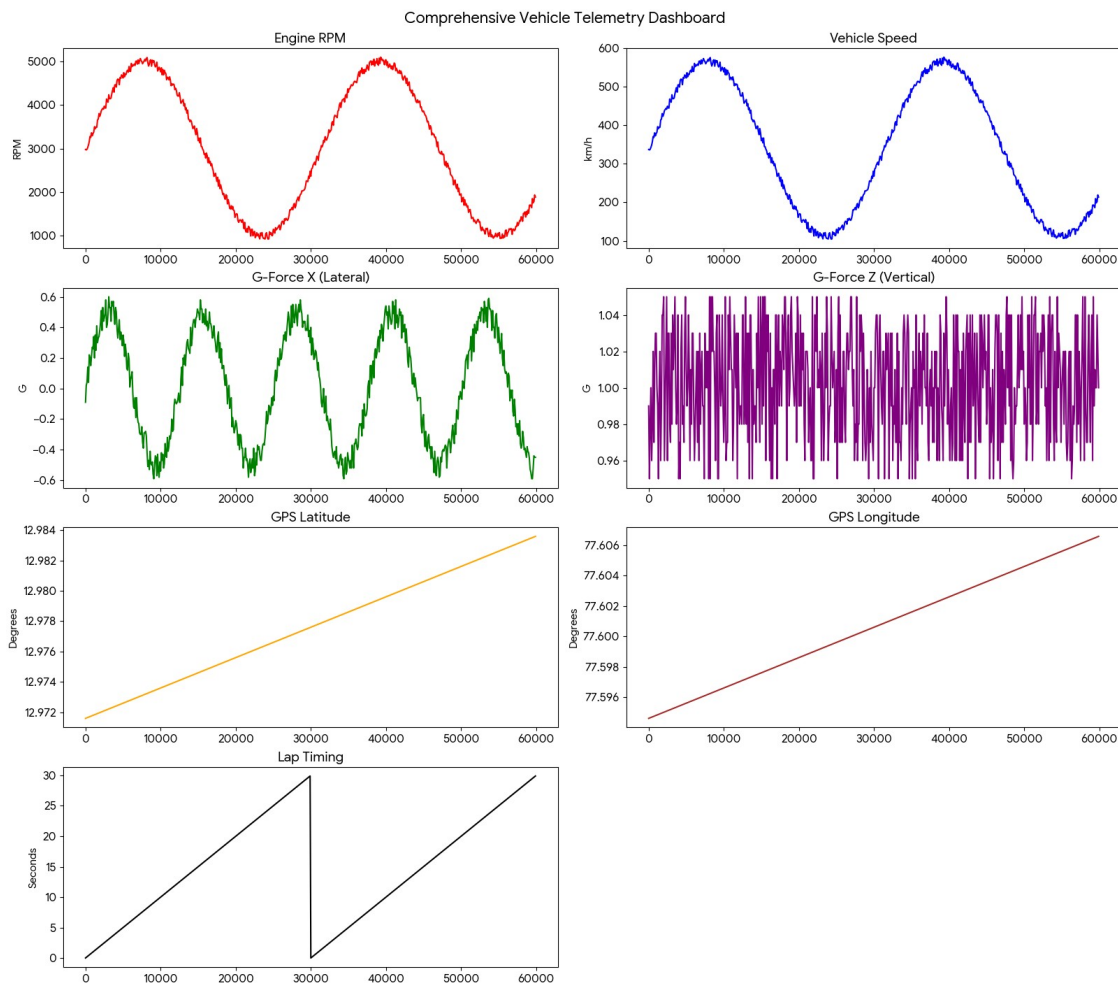
- GPS Longitude
- Lap Timing
- Fault Status

9.2 Telemetry Dashboard Results

The telemetry dashboard demonstrates:

- Engine RPM variation
- Vehicle speed variation
- Lateral G-force changes
- Vertical vibration analysis
- GPS coordinate tracking
- Lap timing analysis

The generated telemetry plots validate the behavior of the telemetry processing system.



10. Test Results & Fault Logs

10.1 System Initialization Results

Successful initialization sequence:

- STM32F401RE boot successful
- FreeRTOS kernel started
- SD card mounted
- IMU initialized
- GPS initialized
- CAN transceiver initialized
- ESP32 telemetry initialized
- RTOS tasks started

10.2 Fault Injection Testing

The telemetry system was tested under simulated failure conditions.

Test 1 — IMU Disconnect

Fault Injected:

- I2C bus error simulated

System Response:

- Fault flag raised
- I2C recovery initiated
- Sensor communication restored

Result:

- System recovered successfully

Test 2 — GPS Signal Loss

Fault Injected:

- GPS timeout simulation

System Response:

- GPS loss fault raised
- Dead reckoning fallback enabled

Result:

- Telemetry continued using alternative estimation

Test 3 — SD Card Write Failure

Fault Injected:

- SD write failure simulated

System Response:

- Fault flag raised
- SD remount attempted
- Buffer flushed after recovery

Result:

- Logging restored successfully

Test 4 — CAN Timeout

Fault Injected:

- CAN message timeout simulated

System Response:

- CAN timeout alarm raised
- RPM data marked stale

Result:

- Fault correctly detected

11. Results

The telemetry system successfully demonstrated:

- Real-time RTOS scheduling
- Multi-sensor telemetry acquisition
- CSV telemetry logging
- Fault monitoring and recovery
- Wireless telemetry streaming support
- Real-time dashboard generation

The FreeRTOS architecture provided deterministic scheduling and modular task separation.

12. Challenges Faced

Hardware Challenges

- CAN integration on STM32F401RE required external MCP2515 controller
- Sensor synchronization and timestamping
- Managing concurrent task timing

Software Challenges

- Queue synchronization
- Fault recovery handling
- Logging reliability
- RTOS task prioritization

13. Future Improvements

Future enhancements planned:

- Real CAN bus integration with ECU
- Advanced sensor fusion algorithms
- Wireless dashboard application

- MQTT/cloud telemetry
- SD card ring-buffer logging
- OTA firmware updates
- BLE mobile application support

14. AI Usage and Methodology

To optimize the development lifecycle, a multi-modal AI toolchain was utilized for design, documentation, and logic validation:

- **Code Assistance (GitHub Copilot):** Used for generating boilerplate code, such as HAL initialization structures and standard I/O driver templates, which significantly reduced initial setup time.
- **Architectural Ideation (ChatGPT/Claude):** Leveraged for high-level RTOS task scheduling, inter-process communication (IPC) design, and conceptualizing robust fault-monitoring strategies.
- **Technical Research & Verification (Perplexity AI):** Employed as an accelerated research tool to cross-reference datasheet specifications (e.g., MPU6050 register maps, SPI timing constraints, and CAN-bus protocol details) against the STM32F401RE Reference Manual.
- **Diagrammatic Logic (Mermaid.js/Eraser.io):** AI-assisted text-to-diagram generation was utilized to define the architecture diagrams and data flow logic, ensuring clear visual representation of task dependencies.
- **Documentation & Refinement:** AI tools were used to format technical reports, refine engineering terminology, and ensure consistent documentation standards.

Human-in-the-Loop Statement: The integration of these tools served as a force-multiplier for development efficiency. However, all firmware implementation, hardware integration, and peripheral testing were performed manually. All AI-generated code snippets and architectural suggestions were subjected to critical review, modification, and strict validation against the

STM32 hardware constraints, memory limits, and timing requirements of the target platform. No autonomous code generation was committed to the production repository without thorough verification.

15. References

- [1] **STMicroelectronics**. (2023). *STM32F401xD/xE Reference Manual (RM0368)*. [Online]. Available: [st.com](https://www.st.com)
- [2] **Amazon Web Services**. (2024). *FreeRTOS Kernel Developer Guide*. [Online]. Available: freertos.org
- [3] **ChaN**. (2024). *FatFs - Generic FAT Filesystem Module*. [Online]. Available: elm-chan.org
- [4] **InvenSense/TDK**. (2013). *MPU-6000 and MPU-6050 Product Specification (Revision 3.4)*. Technical Datasheet.
- [5] **u-blox**. (2011). *NEO-6 u-blox 6 GPS Modules Data Sheet (Doc. No. GPS.G6-HW-09005)*. Technical Datasheet.
- [6] **Microchip Technology Inc**. (2007). *MCP2515 Stand-Alone CAN Controller with SPI Interface (DS21801E)*. Technical Datasheet.
- [7] **Espressif Systems**. (2023). *ESP32 Series Datasheet (v4.3)*. [Online]. Available: [espressif.com](https://www.espressif.com)
- [8] **STMicroelectronics**. (2024). *STM32CubeIDE User Guide (UM2609)*. [Online]. Available: [st.com](https://www.st.com)

16. Conclusion

The successful completion of this project validates the design methodology and implementation of a real-time embedded telemetry and data logging system utilizing the STM32F401RE microcontroller and FreeRTOS. By employing a modular, multi-threaded firmware architecture, the system achieves robust concurrent data acquisition, reliable SD card logging, wireless telemetry streaming, and deterministic fault monitoring.

This project underscores key engineering competencies required for modern embedded systems, including:

- **Real-Time Scheduling:** Managing task priorities and timing determinism through the FreeRTOS kernel.
- **Inter-Process Communication:** Implementing thread-safe data exchange using queues and mutexes to ensure system stability.
- **Embedded C Development:** Writing efficient, low-level drivers and leveraging HAL-based peripheral management.
- **Fault-Tolerant Design:** Engineering autonomous "self-healing" mechanisms to ensure system resilience against hardware disconnects and communication failures.
- **Data Processing & Storage:** Transforming high-frequency raw sensor inputs into actionable telemetry streams while ensuring data integrity in persistent storage.

The resulting architecture provides a scalable and robust foundation for advanced automotive and motorsport telemetry applications. It effectively demonstrates that a disciplined, architecture-first approach to RTOS-based design can successfully bridge the gap between simple sensor polling and high-reliability, mission-critical embedded systems.